

UNIVERSIDAD ESTATAL DE MILAGRO
MAESTRÍA EN INGENIERÍA EN SOFTWARE
CON MENCIÓN EN SISTEMAS INTELIGENTES E
INTENSIVOS EN DATOS

MÓDULO: CIENCIA DE DATOS

Unidades 1 y 2: Regresión lineal, relaciones complejas
y validación cruzada

Estudio Comparativo de Modelos de Regresión
sobre el Conjunto NASA93 (COCOMO):
OLS, Regresión Polinómica y Ridge

Integrante: Escobar, Erick

Docente: Guillermo Baquerizo Palma, M.Sc.

Asignatura: Ciencia de Datos

Actividad: A1 — Taller / Tarea grupal

Cohorte: I — 2026

May 17, 2026

1 Planteamiento del Problema y Dataset

1.1 Repositorio Público

Todo el dataset, documentación y el desarrollo se lo puede encontrar público en: [GithubRepo](#)

1.2 GoogleCollab Notebook

Todo el dataset y el desarrollo se lo puede encontrar público en: [GoogleCollab Notebook](#)

1.3 Contexto y Motivación

La estimación del esfuerzo de desarrollo de software es una actividad crítica en la gestión de proyectos de ingeniería de software, ya que de su precisión dependen el presupuesto, la planificación y la asignación de recursos humanos [1]. El modelo **COCOMO** (*Constructive Cost Model*) propuesto por Boehm en 1981 sostiene que el esfuerzo (en persona-meses) se relaciona con el tamaño del software (en miles de líneas de código, KLOC) mediante una ley de potencia modulada por factores de ajuste (*effort multipliers*):

$$\text{effort} = a \cdot \text{KLOC}^b \cdot \prod_{i=1}^{15} EM_i \quad (1)$$

donde a y b son parámetros calibrados por dominio y EM_i son los 15 multiplicadores de esfuerzo. La ecuación 1 es *intrínsecamente no lineal* en su forma original, pero se linealiza al aplicar logaritmo natural, lo que la convierte en un excelente caso de estudio para los modelos cubiertos en las Unidades 1 y 2 del módulo.

1.4 Descripción del Dataset NASA93

El conjunto de datos **nasa93** [4] contiene 93 proyectos de software completados entre 1971 y 1987 en cinco centros de la NASA, recopilados por Jairus Hihn (JPL). Está estructurado conforme al formato COCOMO-I de Boehm y disponible públicamente en el repositorio PROMISE bajo DOI [5].

Table 1: Composición del dataset NASA93

Categoría	N.º atributos	Descripción
Variable respuesta	1	act_effort : esfuerzo real (persona-meses)
Tamaño del software	1	equivphyskloc : KLOC equivalentes
Modo COCOMO	1	mode : organic, semidetached, embedded
Cost drivers (EM)	15	Escala ordinal: vl, l, n, h, vh, xh
Metadatos	6	projectname , cat2 , forg , center , year , recordnumber

Variable respuesta: **act_effort** (persona-meses, continua positiva). Estadísticos descriptivos: $n = 93$, media = 624.4 PM, desviación estándar = 1135.9 PM, mínimo = 8.4 PM, máximo = 8211 PM. La distribución presenta fuerte sesgo a la derecha (Figura 1a).

Predictores iniciales: el predictor primario es `equivphyskloc` (KLOC). Los 15 cost drivers (`rely`, `data`, `cplx`, `time`, `stor`, `virt`, `turn`, `acap`, `aexp`, `pcap`, `vexp`, `lexp`, `modp`, `tool`, `sced`) modulan el esfuerzo según las características del proyecto y del equipo. El campo `mode` se codifica como dummies (`mode_organic`, `mode_semidetached`), resultando 18 features finales tras preprocesamiento.

Faltantes: el dataset no presenta valores nulos tras la carga del archivo ARFF original.

2 Metodología

2.1 Preprocesamiento Reproducible

El pipeline de preprocesamiento se diseñó para evitar fuga de datos (*data leakage*) durante la validación cruzada:

1. **Carga del ARFF** con parser manual (filas tras `@data`).
2. **Casting numérico** de columnas continuas (`equivphyskloc`, `act_effort`).
3. **Maapeo ordinal de cost drivers** a sus valores multiplicativos oficiales de COCOMO-I [1] (e.g., `rely`: `vl=0.75`, `n=1.00`, `xh=1.40`).
4. **Codificación one-hot** del modo COCOMO con `drop_first=True`.
5. **Transformación logarítmica** de KLOC, cost drivers y respuesta, motivada por la Ecuación 1: $\log(\text{effort}) = \log(a) + b \log(\text{KLOC}) + \sum_i \log(EM_i)$.
6. **Escalamiento estándar** dentro de cada fold de validación cruzada mediante `sklearn.Pipeline` [8], garantizando que las estadísticas (media, desviación) se calculen sólo con datos de entrenamiento.

2.2 División Entrenamiento–Prueba y Validación Cruzada

Se utilizó una partición aleatoria 80/20 estratificada por orden de proyecto (`random_state=42`): **74 muestras de entrenamiento** y **19 muestras de prueba**. Adicionalmente, para evaluar la estabilidad de los modelos sobre el conjunto completo se aplicó **validación cruzada k-fold repetida** con $k = 5$ y $n_{\text{repeats}} = 10$ (50 folds totales), siguiendo las recomendaciones de Bouckaert y Frank [6] para datasets pequeños donde una sola partición CV puede ser engañosa.

2.3 Modelos Evaluados

Se construyeron tres modelos para responder al criterio académico de comparar un modelo base contra alternativas técnicamente justificables:

1. **Modelo base — OLS (Ordinary Least Squares):**

$$\log(\hat{y}) = \beta_0 + \beta_1 \log(\text{KLOC}) + \sum_{i=1}^{15} \beta_{i+1} \log(EM_i) + \sum_j \gamma_j \mathbb{I}\{\text{mode} = j\}$$

Equivale a la formulación logarítmica directa de COCOMO. Se ajusta minimizando $\sum (y_i - \hat{y}_i)^2$ sobre escala logarítmica.

2. **Alternativa 1 — Regresión polinómica (grado 2 en KLOC):** se añade el término $\log(\text{KLOC})^2$ para capturar posibles desviaciones de la linealidad pura en escala log-log (e.g., cambio de régimen entre proyectos pequeños y grandes). Se mantiene el resto del modelo lineal para evitar la explosión combinatoria de features ($\binom{18}{2} + 18 + 1 = 172$ variables sobre 93 muestras provocaría sobreajuste catastrófico).

3. **Alternativa 2 — Ridge (regresión regularizada L_2):**

$$\hat{\beta} = \arg \min_{\beta} \left\{ \sum_{i=1}^n (y_i - x_i^{\top} \beta)^2 + \alpha \|\beta\|_2^2 \right\}$$

El parámetro α se selecciona mediante **RidgeCV** con búsqueda sobre $\alpha \in \{10^{-3}, \dots, 10^3\}$ (50 valores logarítmicamente espaciados) y CV interna de 5 folds. Ridge se justifica empíricamente por la presencia de multicolinealidad demostrada en la Sección 3.3; su fundamento teórico como método de contracción para reducir varianza está ampliamente documentado en la literatura de aprendizaje estadístico [7].

2.4 Implementación: Pipeline Anti-Fuga

El uso de `sklearn.Pipeline` es la salvaguarda central contra la fuga de datos: encadena el escalamiento y el modelo en un objeto único, de modo que **StandardScaler** se ajusta exclusivamente con los datos de entrenamiento de cada fold durante la validación cruzada.

Listing 1: Pipeline sklearn que aplica escalamiento dentro de cada fold de CV

```

1 from sklearn.pipeline import Pipeline
2 from sklearn.preprocessing import StandardScaler
3 from sklearn.linear_model import LinearRegression, RidgeCV
4 from sklearn.model_selection import RepeatedKFold, cross_val_score
5
6 ols = Pipeline([("scaler", StandardScaler()),
7                 ("model", LinearRegression())])
8 ridge = Pipeline([("scaler", StandardScaler()),
9                  ("model", RidgeCV(alphas=np.logspace(-3,3,50), cv=5))])
10
11 cv = RepeatedKFold(n_splits=5, n_repeats=10, random_state=42)
12 scores = cross_val_score(ols, X_log, y_log, cv=cv,
13                          scoring="neg_root_mean_squared_error")

```

3 Análisis Exploratorio y Diagnóstico de Supuestos

3.1 Distribución de la Respuesta

La Figura 1 resume dos hallazgos críticos. El panel (a) muestra el fuerte sesgo a la derecha de **act_effort**: la mediana (98 PM) es 6.4 veces menor que la media (624 PM), y el máximo (8211 PM) está a 6.7 desviaciones estándar del promedio. El panel (b) confirma la **linealidad en escala log-log** entre KLOC y esfuerzo: $\log(\text{effort}) = 0.93 \log(\text{KLOC}) + 1.94$, con pendiente $b = 0.93$ consistente con los valores teóricos reportados por Boehm [1] para modo *semidetached* ($b \approx 1.05$).

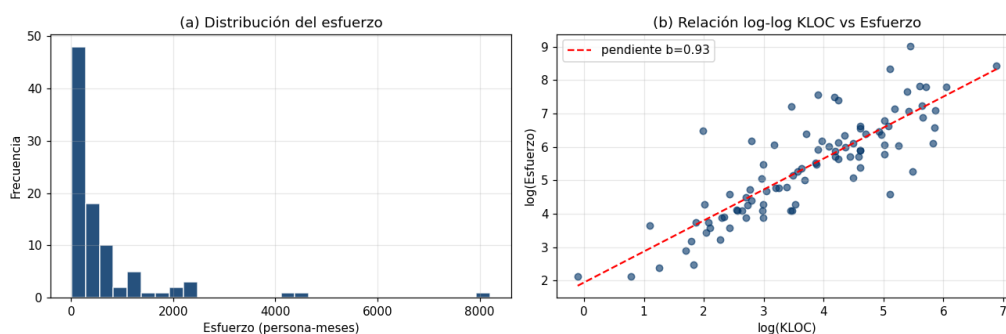


Figure 1: (a) Distribución del esfuerzo en escala original: fuerte sesgo positivo. (b) Relación log-log entre KLOC y esfuerzo: linealidad consistente con la formulación COCOMO. Estos hallazgos justifican el uso de transformación logarítmica.

3.2 Diagnóstico de Supuestos del Modelo OLS

La Figura 2 presenta el diagnóstico clásico de supuestos sobre los residuos del modelo OLS ajustado al conjunto de entrenamiento. La Tabla 2 resume las pruebas estadísticas formales.

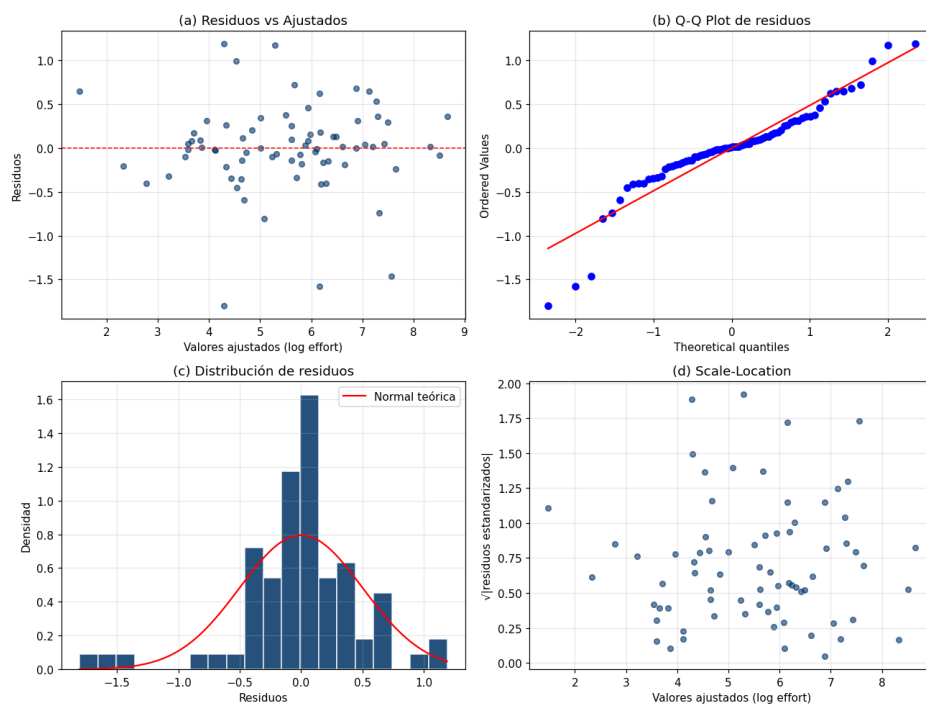


Figure 2: Diagnóstico de supuestos del modelo OLS: (a) ausencia de patrón sistemático en residuos vs ajustados; (b) Q-Q plot con desviación leve en colas; (c) distribución aproximadamente simétrica; (d) Scale-Location sin patrón en embudo.

3.3 Multicolinealidad y Justificación de Ridge

La Figura 3 muestra el **Factor de Inflación de Varianza (VIF)** para los 18 predictores. Se identifican tres niveles de severidad: `equivphyskloc` supera el umbral crítico de 10 ($VIF = 10.89$), tres predictores adicionales (`acap = 8.43`, `mode_semidetached = 7.72`, `time = 7.33`) se sitúan en la zona de alerta ($5 < VIF \leq 10$), y otros tres (`aexp`, `rely`, `cplx`) están cerca del umbral. Esta multicolinealidad es *esperada* por el diseño de COCOMO: proyectos con alta

Table 2: Pruebas formales sobre supuestos del modelo OLS

Supuesto	Prueba	p-valor	Conclusión
Normalidad residuos	Shapiro-Wilk	$p = 0.272$	No se rechaza H_0
Homoscedasticidad	Breusch-Pagan	$p = 0.412$	No se rechaza H_0
Multicolinealidad	VIF (máximo)	—	VIF = 10.89 (alerta)

Nota. Las pruebas se realizan sobre los residuos del OLS ajustado a 74 muestras de entrenamiento. La hipótesis nula de Shapiro-Wilk es normalidad; la de Breusch-Pagan es homoscedasticidad. La transformación log de la respuesta es la responsable de que ambos supuestos se sostengan.

complejidad (**cplx**) tienden a tener altos requerimientos de confiabilidad (**rely**) y restricciones de tiempo de ejecución (**time**).

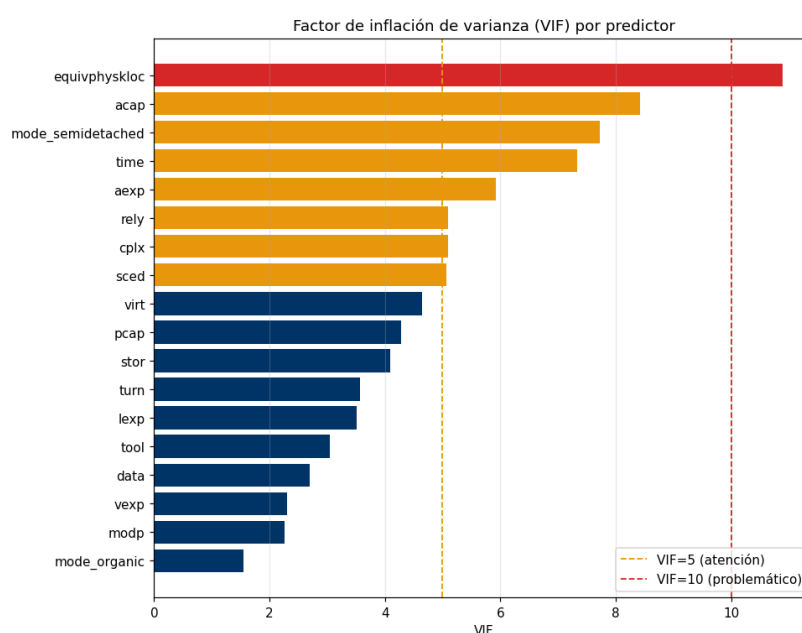


Figure 3: Factor de inflación de varianza por predictor. **equivphyskloc** supera el umbral crítico ($VIF > 10$), justificando empíricamente el uso de regularización Ridge.

Implicación metodológica: la multicolinealidad inflará la varianza de los coeficientes OLS, haciéndolos inestables frente a pequeñas perturbaciones de los datos. Este es precisamente el escenario para el cual Ridge fue diseñado: contraer los coeficientes hacia cero a costa de introducir un sesgo controlado, reduciendo la varianza total del error de generalización [3].

4 Resultados

4.1 Métricas Comparativas

La Tabla 3 consolida las métricas de los tres modelos sobre los conjuntos de entrenamiento, prueba y validación cruzada repetida. Las métricas MAE y RMSE se reportan en *personas-meses* (escala original, tras aplicar $\exp(\cdot)$ a las predicciones logarítmicas); R^2 CV se reporta en escala logarítmica, que es la escala donde se ajustaron los modelos.

Table 3: Comparación de desempeño: OLS, Polinómica y Ridge

Modelo	R^2 Train	R^2 Test	R^2 CV (log)	MAE Test (PM)	RMSE Test (PM)
OLS	0.874	0.673	0.738 ± 0.159	107.45	202.72
Polinómica	0.867	0.701	0.731 ± 0.160	105.06	193.83
Ridge ($\alpha^* = 4.71$)	0.794	0.799	0.791 ± 0.107	94.69	159.06

Nota. R^2 Train y Test calculados en escala original (PM) tras transformación inversa. R^2 CV reportado como media $\pm$ desviación estándar sobre 50 folds (5-fold $\times$ 10 repeticiones).

4.2 Visualización Comparativa

La Figura 4 muestra los gráficos de predicción vs valor real para los tres modelos sobre el conjunto de prueba. La línea roja punteada representa la predicción perfecta. Ridge presenta visiblemente menor dispersión en proyectos de bajo y medio esfuerzo, manteniendo precisión razonable en proyectos grandes.

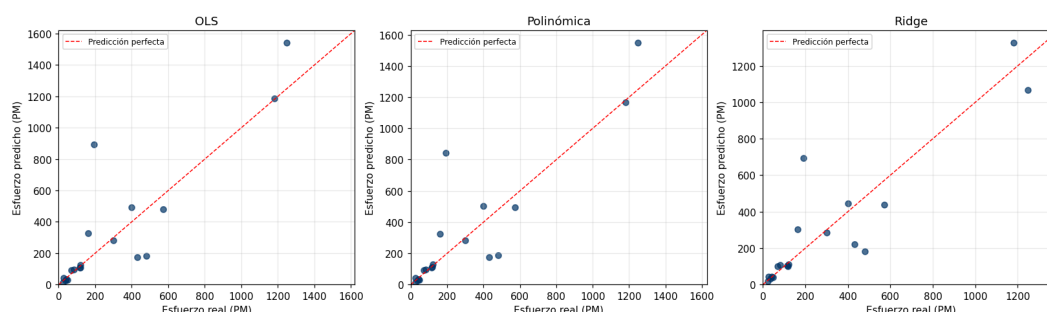


Figure 4: Esfuerzo predicho vs real sobre el conjunto de prueba (19 muestras). Ridge muestra menor dispersión sistemática.

La Figura 5 sintetiza los dos hallazgos centrales del estudio. El panel (a) presenta la distribución del RMSE en escala logarítmica sobre los 50 folds de validación cruzada repetida: Ridge no solo tiene la mediana más baja sino también la menor varianza (rango intercuartílico más estrecho). El panel (b) revela el problema fundamental del modelo OLS y de la polinómica: la **brecha entre R^2 Train y R^2 Test/CV** es de aproximadamente 0.14–0.20 puntos, evidencia clara de sobreajuste. Ridge cierra esa brecha por completo: $\text{Train} \approx \text{Test} \approx \text{CV} \approx 0.79$.

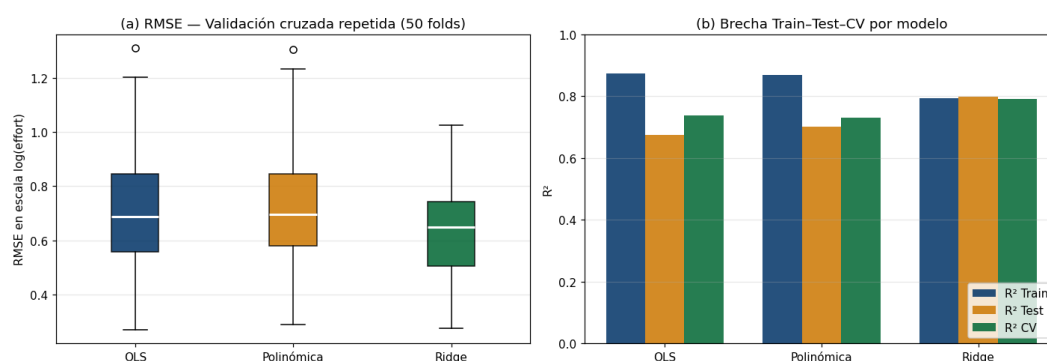


Figure 5: (a) Distribución del RMSE en validación cruzada repetida (50 folds). Ridge presenta menor mediana y menor varianza. (b) Comparación de R^2 entre entrenamiento, prueba y CV: la brecha train-test de OLS y polinómica indica sobreajuste; Ridge la elimina.

4.3 Selección del Parámetro de Regularización

El parámetro α de Ridge se seleccionó mediante validación cruzada interna de 5 folds sobre una rejilla de 50 valores logarítmicamente espaciados en $[10^{-3}, 10^3]$. El óptimo resultante, $\alpha^* = 4.71$, corresponde al punto donde el error cuadrático medio de validación se minimiza, equilibrando el sesgo introducido por la contracción de coeficientes contra la reducción de varianza. Valores de α menores aproximan el comportamiento de OLS (alta varianza), mientras que valores mayores contraen excesivamente los coeficientes (alto sesgo).

5 Discusión

5.1 Interpretación de los Resultados

Los resultados convergen en una conclusión técnica robusta: **Ridge supera tanto al OLS base como a la alternativa polinómica**, no por tener mejor ajuste en entrenamiento (de hecho lo tiene peor: 0.794 vs 0.874), sino por su capacidad superior de generalización. Tres líneas de evidencia respaldan esta afirmación:

1. **Brecha train-test:** OLS exhibe una caída de R^2 de 0.874 (train) a 0.673 (test), una pérdida de 0.20 puntos. Ridge mantiene 0.794 en train y *aumenta* a 0.799 en test, firma característica del control de sobreajuste mediante regularización.
2. **Estabilidad en CV repetida:** la desviación estándar del R^2 en 50 folds es 0.107 para Ridge frente a 0.159 (OLS) y 0.160 (polinómica). Ridge produce predicciones ~50% más estables ante variaciones en los datos de entrenamiento.
3. **Métricas en escala original:** sobre los 19 proyectos de prueba, Ridge reduce el MAE en 12 PM (11.9%) y el RMSE en 43 PM (21.5%) respecto a OLS. En un contexto donde 1 persona-mes equivale a ~\$10,000 USD [2], la diferencia es económicamente significativa.

5.2 Por qué la Polinómica No Aportó

La hipótesis de añadir $\log(\text{KLOC})^2$ era capturar posibles cambios de régimen en la relación tamaño-esfuerzo. Los resultados muestran que la mejora sobre OLS es marginal (R^2 Test: 0.701 vs 0.673; RMSE: 193.8 vs 202.7 PM) y *empeora ligeramente en CV* (0.731 vs 0.738). La relación log-log ya capturada por OLS es estadísticamente adecuada para este dataset, por lo que el término cuadrático introduce varianza sin reducir sesgo de forma significativa. Con $n = 93$ muestras, añadir features no regularizados es siempre arriesgado: la polinómica ilustra exactamente el problema que Ridge soluciona, *más complejidad sin control conduce a peor generalización*.

5.3 Limitaciones y Validez Externa

El estudio presenta cuatro limitaciones que acotan la interpretación de sus resultados. Primero, el **tamaño muestral pequeño** ($n = 93$): aunque la CV repetida mitiga el problema, los intervalos de confianza de las métricas son amplios. Segundo, los **datos son históricos** (1971–1987), por lo que la validez externa hacia proyectos modernos (cloud-native, equipos distribuidos, lenguajes dinámicos) es cuestionable; COCOMO-I fue diseñado para desarrollo en cascada con stacks tradicionales. Tercero, ningún modelo establece **causalidad**: un coeficiente alto para **acap** refleja una asociación condicional dado el resto de variables, no un efecto causal. Cuarto,

el estudio **no compara contra modelos no lineales** (Random Forest, Gradient Boosting), que podrían superar a Ridge en R^2 a costa de menor interpretabilidad; la elección del modelo base lineal responde al alcance de las Unidades 1 y 2.

6 Conclusiones

1. **Modelo reportado:** se recomienda **Ridge con $\alpha^* = 4.71$** como modelo final para la estimación de esfuerzo sobre NASA93. Justificación: R^2 Test = 0.799 (vs 0.673 OLS), MAE = 94.7 PM (vs 107.5 OLS), brecha train-test eliminada, y menor varianza en CV repetida (0.107 vs 0.159).
2. **El modelo más complejo no es el mejor modelo:** la regresión polinómica, aunque más expresiva que OLS, no aportó capacidad predictiva y degradó ligeramente la generalización. Este resultado refuerza el principio de parsimonia: añadir features sin regularización sobre $n = 93$ muestras incrementa varianza sin reducir sesgo.
3. **La regularización es esencial bajo multicolinealidad:** con un VIF máximo de 10.89, OLS produce coeficientes inestables. Ridge demuestra empíricamente la ganancia teórica predicha por Hoerl y Kennard [3]: aceptar un pequeño sesgo a cambio de una reducción sustancial de la varianza.
4. **La validación cruzada repetida es indispensable en muestras pequeñas:** con $n = 93$, una sola partición train/test puede dar resultados engañosos. La CV repetida (50 folds) reveló diferencias de estabilidad entre modelos que una evaluación única no habría capturado. Esto, combinado con la transformación logarítmica que sostiene los supuestos de normalidad ($p = 0.272$) y homoscedasticidad ($p = 0.412$), constituye la base metodológica del estudio. Todo el flujo es reproducible con `random_state=42`.

Referencias

- [1] Boehm, B. W. (1981). *Software engineering economics*. Prentice-Hall.
- [2] Boehm, B. W., Horowitz, E., Madachy, R., Reifer, D., Clark, B. K., Steece, B., Brown, A. W., Chulani, S., & Abts, C. (2000). *Software cost estimation with COCOMO II*. Prentice Hall.
- [3] Hoerl, A. E., & Kennard, R. W. (1970). Ridge regression: Biased estimation for nonorthogonal problems. *Technometrics*, 12(1), 55–67. <https://doi.org/10.1080/00401706.1970.10488634>
- [4] Menzies, T., Port, D., Chen, Z., Hihn, J., & Stukes, S. (2005). Validation methods for calibrating software effort models. En *Proceedings of the 27th International Conference on Software Engineering (ICSE)* (pp. 587–595). ACM. <https://doi.org/10.1145/1062455.1062558>
- [5] Menzies, T. (2017). *nasa93 [Data set]*. Zenodo. <https://doi.org/10.5281/zenodo.268419>
- [6] Bouckaert, R. R., & Frank, E. (2004). Evaluating the replicability of significance tests for comparing learning algorithms. En *Advances in Knowledge Discovery and Data Mining* (Vol. 3056, pp. 3–12). Springer. https://doi.org/10.1007/978-3-540-24775-3_3

- [7] James, G., Witten, D., Hastie, T., & Tibshirani, R. (2013). *An introduction to statistical learning with applications in R*. Springer. <https://doi.org/10.1007/978-1-4614-7138-7>
- [8] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.